2.1 GRAMMATICHE CONTEXT-FREE

Un esempio di grammatica context-free, che chiamiamo G_1, è la seguente:

stringa

$A \rightarrow 0A1$

$A \rightarrow B$

$B \rightarrow \#$

Una grammatica consiste di un insieme di regole di sostituzione, anche chiamate produzioni. Ogni regola appare come una linea della grammatica, costituita da un simbolo e una stringa separati da una freccia. Il simbolo è chiamato una variabile. La stringa consiste di variabili e altri simboli chiamati terminali. Le variabili sono spesso rappresentate da lettere maiuscole. I terminali sono analoghi ai simboli dell'alfabeto di input e sono spesso rappresentati da lettere minuscole, numeri o simboli speciali.

Una sequenza di sostituzioni per ottenere una stringa è chiamata una derivazione. Per esempio, una derivazione della stringa 000#111 nella grammatica G_1 è:

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

Tutte le stringhe generate in questo modo costituiscono il linguaggio della grammatica. Ogni linguaggio che può essere generato da una grammatica context-free è chiamato un linguaggio context-free (CFL).

DEFINIZIONE FORMALE DI GRAMMATICA CONTEXT-FREE

Definizione 2.2

Una grammatica context-free è una quadrupla (V, Σ, R, S), dove:

1. V è un insieme finito i cui elementi sono chiamati variabili;
2. Σ è un insieme finito, disgiunto da V, i cui elementi sono chiamati terminali;
3. R è un insieme finito di regole;
4. $S \in V$ è la variabile iniziale;

2.2 AUTOMI A PILA

In questa sezione introduciamo un nuovo tipo di modello computazionale chiamato automa a pila (pushdown automata). Questi automi sono come gli automi finiti non deterministici ma hanno una componente in più chiamata pila (stack). La pila consente a tali automi di riconoscere alcuni linguaggi non regolari.

Un automa a pila (PDA) può scrivere simboli nella pila e rileggerli in seguito. Scrivere un simbolo "spinge giù" tutti gli altri simboli nella pila. In un qualunque momento il simbolo sulla cima (top) della pila può essere letto e rimosso. I rimanenti simboli allora ritornano più in alto. L'operazione di scrivere un simbolo sulla pila è chiamata push, quella di eliminare un simbolo dalla pila è spesso chiamata pop. Una pila è un dispositivo "last in, first out".

DEFINIZIONE FORMALE DI AUTOMA A PILA

La pila è un dispositivo che contiene simboli presi da un qualche alfabeto. La macchina può usare differenti alfabeti per il suo input e la sua pila, quindi ora specifichiamo sia un alfabeto di simboli di input Σ sia un alfabeto di pila Γ.

Il dominio della funzione di transizione è $Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon$. Quindi lo stato corrente, il prossimo simbolo di input letto e il simbolo sulla cima della pila determinano la mossa seguente di un automa a pila. Per quanto riguarda il codominio della funzione di transizione, dobbiamo considerare cosa fa l'automa quando è in una specifica situazione. Esso può trovarsi in un nuovo stato ed eventualmente scrivere un simbolo sulla cima della pila. La funzione δ può indicare quest'azione restituendo un elemento di Q e un elemento di Γ_ε, quindi un elemento di $Q \times \Gamma_\varepsilon$. Poiché permettiamo il non determinismo in questo modello, una situazione può avere diverse mosse successive lecite. Dunque la funzione δ restituisce $P(Q \times \Gamma_\varepsilon)$.

Definizione 2.13

Un <u>automa a pila</u> è una sestupla $(Q, \Sigma, \delta, q_0, F)$, dove Q, Σ, Γ ed F sono tutti insiemi finiti e

1. Q è l'insieme degli stati;
2. Σ è l'alfabeto di input;
3. Γ è l'alfabeto della pila;
4. $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow P(Q \times \Gamma_\varepsilon)$ è la funzione di transizione;
5. $q_0 \in Q$ è lo stato iniziale;
6. $F \subseteq Q$ è l'insieme degli stati accettanti.

DEFINIZIONE FORMALE DI COMPUTAZIONE DI UN PDA

Siano dati:

- Un automa a pila $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$;
- Una stringa di ingresso w tale che $w = w_1 w_2 \dots w_m$, dove ogni w_i fa parte dell'alfabeto Σ_ε;
- Una stringa s tale che $s = s_1 s_2 \dots s_m$, dove ogni s_i appartiene allo stack;

Diciamo allora che M accetta w se esiste una sequenza di stati $r_0, r_1, \dots, r_m \in Q$ che rispettino tre condizioni:

1. $r_0 = q_0$ e $s_0 = \varepsilon$ (la macchina parte dallo stato iniziale e con lo stack vuoto);
2. Per ogni $i = 0, \dots, m-1$, abbiamo che $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$, dove $s_i = at$ e $s_{i+1} = bt$ per qualche a e $b \in \Gamma_\varepsilon$ e con $t \in \Gamma^*$ (lo stato futuro di M dipende dallo stato attuale, dal simbolo in ingresso e dallo stack);
3. $r_m \in F$ (alla fine dell'input M si trova in uno stato accettante).

EQUIVALENZA CON LE GRAMMATICHE CONTEXT-FREE

In questa sezione mostriamo che grammatiche context-free e automi a pila sono computazionalmente equivalenti. Mostreremo come trasformare ogni grammatica context-free in un automa a pila che riconosce lo stesso linguaggio e viceversa.

Teorema 2.20

Un linguaggio è context-free se e solo se esiste un automa a pila che lo riconosce.

Come al solito, per i teoremi "se e solo se", abbiamo due enunciati da provare, nelle due direzioni.

Lemma 2.21

Se un linguaggio è context-free, allora esiste un automa a pila che lo riconosce.

IDEA: Dato che per definizione un CFL è generato da una grammatica context-free, non dobbiamo fare altro che dimostrare che ogni CFG G può essere convertita in un equivalente PDA P. In pratica dobbiamo costruire P in modo che se gli passiamo in ingresso una stringa w generata da G, questi dovrà accettarla. Ma come fa a capire se è davvero generata dalla grammatica a lui equivalente?

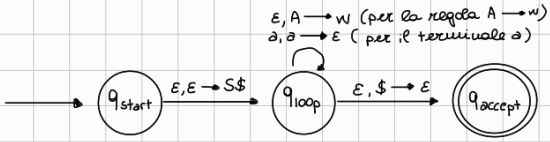
Nel dettaglio, P comincia la sua computazione scrivendo la variabile iniziale sullo stack; poi prosegue attraverso una serie di stringhe intermedie ottenute sostituendo le variabili secondo le regole della grammatica; infine quando arriva a una stringa composta da soli terminali la confronta con w: se corrispondono allora P accetta, altrimenti abbandona quel ramo della computazione.

Ma dove le memorizzo le stringhe intermedie? Non posso metterle così come sono sullo stack. Se infatti dovessi fare una sostituzione di una variabile in una stringa intermedia e questa non si trova in cima allo stack, sarebbe un problema. La soluzione è conservare solo una parte della stringa intermedia nello stack, quella che va dalla prima variabile in poi: tutti i simboli che vengono prima devono corrispondere con la stringa in ingresso, altrimenti buttiamo via tutto.

DIM: Sia dato un automa a pila $P = (Q, \Sigma, \Gamma, \delta, q_1, F)$. I tre stati fondamentali di P sono $Q = \{q_{start}, q_{loop}, q_{accept}\}$. Diamo ora una descrizione del funzionamento di P:

1. $\delta(q_{start}, \varepsilon, \varepsilon) = \{(q_{loop}, S\$)\}$ (metti il simbolo \$ e la variabile iniziale nello stack, quindi vai in q_{loop})
2. Continua a ripetere i seguenti passi:
   1) $\delta(q_{loop}, \varepsilon, A) = \{(q_{loop}, w) \mid A \to w \text{ è una regola in } R\}$ (Se in cima allo stack c'è una variabile A, seleziona in modo non deterministico una delle regole per A e sostituisci A con la stringa che si trova a destra della regola).
   2) $\delta(q_{loop}, a, a) = \{(q_{loop}, \varepsilon)\}$ (Se in cima allo stack c'è un simbolo terminale a, leggi il prossimo simbolo dall'ingresso e confrontalo con a. Se corrispondono, ripeti; altrimenti abbandona questo ramo della computazione non deterministica)
   3) $\delta(q_{loop}, \varepsilon, \$) = \{(q_{accept}, \varepsilon)\}$ (Se in cima allo stack c'è il simbolo \$, entra nello stato di accettazione).

Ecco il diagramma degli stati di P:



Lemma 2.27

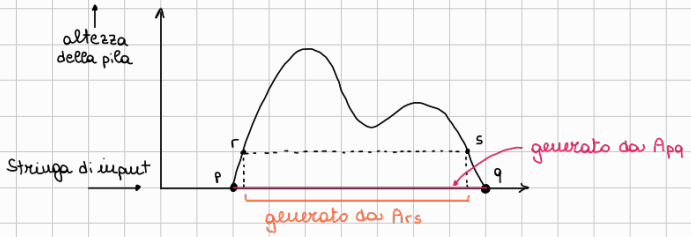
Se un linguaggio è riconosciuto da un automa a pila, allora esso è context-free.

IDEA: Situazione inversa: abbiamo un PDA P e vogliamo creare un CFG G che generi tutte le stringhe accettate da P.
Come prima cosa, dovremo aggiungere alla nostra grammatica G la variabile A_{pq} per ogni coppia di stati p e q in P. Non è una variabile a caso, ma quella che genera tutte le stringhe che possono portare il PDA da p a q lasciando lo stack nelle stesse condizioni in cui lo si è trovato. Per riuscirci dobbiamo fornire a P tre nuove caratteristiche:

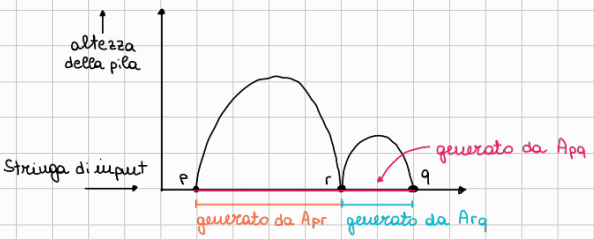
1. Deve avere un unico stato accettante q_{accept};
2. deve svuotare il suo stack prima di accettare;
3. Ogni transizione deve consistere o in un'operazione di push o in una di pop, non entrambe.

Qualcosa sul comportamento di P già lo si capisce da queste caratteristiche: la prima operazione su una qualsiasi stringa x sarà di push, l'altra di pop. Ma le altre? Distinguiamo due casi:

- Il simbolo estratto dallo stack alla fine della computazione è quello che avevo messo all'inizio, quindi lo stack è vuoto solo in questi due momenti. Simuliamo questa eventualità con la regola: $A_{pq} \to a A_{rs} b$, dove a è il simbolo letto in ingresso al primo passo, b è il simbolo letto in ingresso all'ultimo passo, r è lo stato che segue p ed s è quello che precede q.
  Aiutiamoci con uno schema:



- Seconda situazione: il simbolo estratto dallo stack alla fine della computazione non è quello che avevo messo all'inizio, quindi lo stack si è svuotato completamente tra p e q almeno una volta, in uno stato che chiameremo r. Per questa eventualità introduciamo la regola: $A_{pq} \to A_{pr} A_{rq}$. Aiutiamoci anche in questo caso con uno schema:



la costruzione di G termina aggiungendo per ogni $p \in Q$ la regola $A_{pp} \longrightarrow \varepsilon$.
la costruzione è questa ma per dimostrare la seconda implicazione del teorema dovremo dimostrare che A_{pq} genera una stringa x se e solo se 😊 x può portare p da p con stack vuoto a q con stack vuoto. Distinguiamo ancora i due versi dell'implicazione:

Fatto 2.30

Se A_{pq} genera x, allora x può portare P da p con la pila vuota a q con la pila vuota.

DIM: la dimostrazione viene fatta per induzione.

- Passo base: la derivazione ha un solo passo, quindi nella parte destra della regola ci sono solo terminali. L'unica regola di questo tipo in G è $A_{pp} \longrightarrow \varepsilon$, che è verificata perché se non prendo niente in ingresso parto con lo stack vuoto e finisco con lo stack vuoto.
- Passo induttivo: Consideriamo vere le derivazioni con al più k passi, con $k \geq 1$, e verifichiamo se sono vere anche per k+1 passi. Supponiamo di avere $A_{pq} \xrightarrow{*} x$ (A_{pq} deriva x) in k+1 passi. Come abbiamo visto, nel primo passo di derivazione sono due le regole che possono essere applicate:
  1) $A_{pq} \longrightarrow a A_{rs} b$: consideriamo una porzione y della x generata da A_{rs}, tale che $x = ayb$. Dato che $A_{rs} \xrightarrow{*} y$ in k passi, l'ipotesi induttiva ci dice che P può andare da r con stack vuoto ad s con stack vuoto. Vediamo bene cosa succede: P parte da p con lo stack vuoto, e dopo aver letto a va nello stato r mettendo t in cima allo stack; poi p legge la stringa y che lo porta allo stato s lasciando t lì dov'è; infine P legge b che lo porta allo stato q facendogli fare un pop di t dallo stack. Perfetto: parto e arrivo con lo stack vuoto;
  2. $A_{pq} \longrightarrow A_{pr} A_{rq}$: consideriamo le porzioni y e z delle x che generano rispettivamente A_{pr} e A_{rq}, tali che $x = yz$. Dato che $A_{pr} \xrightarrow{*} y$ e $A_{rq} \xrightarrow{*} z$ in massimo k passi, l'ipotesi induttiva ci dice che y può portare P da p a r con lo stack vuoto all'inizio e alla fine; stessa cosa per z che porta P da r a q. Quindi, sommando gli effetti, x può portare P dallo stato p con stack vuoto allo stato q con stack vuoto. Perfetto, abbiamo completato il passo induttivo.

Fatto 2.31

Se x può portare P da p con la pila vuota a q con la pila vuota, A_{pq} genera x.

DIM: Anche in questo caso la dimostrazione va fatta per induzione.

- Passo base: la computazione ha 0 passi, quindi stato iniziale e finale coincidono. Chiamiamo p questo stato, per cui vale la regola $A_{pp} \xrightarrow{*} x$. Ma dato che in 0 passi P può leggere solo la stringa vuota, allora $x = \varepsilon$; sostituiamo il valore di x nella regola di prima e otteniamo $A_{pp} \longrightarrow \varepsilon$, che è una regola di G per costruzione. La base è provata.
- Passo induttivo: consideriamo vere le computazioni con al più k passi, con $k \geq 0$, e verifichiamo se sono vere per k+1 passi. Se supponiamo che P abbia una computazione in cui x porta p a q con stack vuoto in k+1 passi, i casi sono due:
  1) "lo stack è vuoto solo all'inizio e alla fine della computazione": chiamiamo t il simbolo che metto in cima allo stack alla prima mossa e che tolgo solo all'ultima. Sia a l'ingresso letto al primo passo, b quello letto all'ultimo, r lo stato in cui vado dopo la prima mossa, ed s lo stato che precede l'ultima. Abbiamo allora che $(r,t) \in \delta(p,a,\varepsilon)$ e $(q,\varepsilon) \in \delta(s,b,t)$, perciò la regola $A_{pq} \longrightarrow a A_{rs} b$ è in G.

     Se adesso chiamiamo y la parte di x priva dei simboli a e b (dove $x = ayb$), l'ingresso y può portare P dallo stato r allo stato s senza toccare il simbolo t nello stack. Evitando di considerare a e b abbiamo rimosso due passaggi dai k+1 necessari per la computazione di x, quindi y richiede k-1 passi. Grazie all'ipotesi induttiva possiamo dunque affermare che $A_{rs} \xrightarrow{*} y$, e di conseguenza $A_{pq} \xrightarrow{*} x$.

  2) "lo stack è vuoto all'inizio e alla fine della computazione, e anche in un altro punto": sia r lo stato diverso da quello iniziale e finale, in cui lo stack è vuoto. Quindi le parti che vanno da p a r e da r a q impiegano al massimo k passi. Chiamiamo ora y l'ingresso letto da P nella prima parte e z quello letto nella seconda. L'ipotesi induttiva ci dice che $A_{pr} \xrightarrow{*} y$ e $A_{rq} \xrightarrow{*} z$. Ma dato che $A_{pq} \longrightarrow A_{pr} A_{rq}$ è in G, $A_{pq} \xrightarrow{*} x$, e quindi abbiamo completato il passo induttivo.

Corollario 2.32

Ogni linguaggio regolare è context-free.

2.3 LINGUAGGI NON CONTEXT-FREE

In questa sezione presentiamo una tecnica per dimostrare che alcuni linguaggi non sono context-free.

IL PUMPING LEMMA PER I LINGUAGGI CONTEXT-FREE

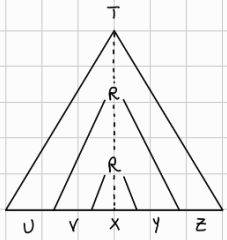
Teorema 2.24

Se A è un linguaggio context-free, allora esiste un numero p (la lunghezza del pumping) tale che, se s è una qualsiasi stringa in A di lunghezza almeno p, allora s può essere divisa in cinque parti $s = uvxyz$ che soddisfano le condizioni:

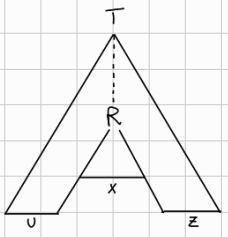
1. per ogni $i \geq 0$, $uv^ixy^iz \in A$
2. $|vy| > 0$
3. $|vxy| \leq p$

Quando s è divisa in $uvxyz$, la condizione 2 afferma che v o y non è la stringa vuota, altrimenti il teorema sarebbe banalmente vero.

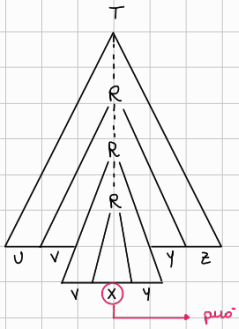
IDEA: Sia A un CFL e sia G un CFG che lo genera. Sia s una stringa "molto lunga" in A. Dato che $s \in A$, essa è derivabile da G e quindi ha un albero sintattico. Poiché s è molto lunga, l'albero sintattico sarà "molto alto".



Per il principio della piccionaia, qualche simbolo di variabile R si deve ripetere lungo il cammino. Questa ripetizione ci permette di sostituire il sottoalbero sotto la seconda occorrenza di R con il sottoalbero sotto la prima occorrenza di R e ottenere ancora un albero sintattico consentito. Pertanto possiamo dividere s in cinque parti $uvxyz$ e possiamo replicare il secondo e quarto pezzo e ottenere una stringa ancora nel linguaggio.



Pumping zero volte



Pumping due volte

può essere anche ε. E' un caso particolare

DIM: Sia G un CFG per il CFL A. Sia b il massimo numero di simboli nel lato destro di una regola (assumiamo che sia almeno 2). Sappiamo che, in ogni albero sintattico costruito usando questa grammatica, un nodo non può avere più di b figli. In altre parole, ci sono al più b foglie in un passo dalla variabile iniziale; ci sono al più b^2 foglie in 2 passi dalla variabile iniziale; e ci sono al più b^h foglie in h passi dalla variabile iniziale. Viceversa, se una stringa generata è almeno b^h+1, ogni dei suoi alberi sintattici deve essere almeno $h+1$ alto.

FORMA NORMALE DI CHOMSKY

La forma normale di Chomsky ci permette di riscrivere una CFG in forma più compatta e semplificata.

Definizione 2.8
Una grammatica context-free è in forma normale di Chomsky se ogni regola è della forma:

$$A \longrightarrow BC$$
$$A \longrightarrow a$$

dove a è un terminale e A, B, C sono variabili qualsiasi - tranne che B e C non possono essere la variabile iniziale. Inoltre, permettiamo la regola $S \longrightarrow \varepsilon$, dove S è la variabile iniziale.

Teorema 2.9
Ogni linguaggio context-free è generato da una grammatica context-free in forma normale di Chomsky.

DIM: Dato che per definizione un CFL è generato da una CFG, non dobbiamo fare altro che dimostrare che ogni CFG G può essere espressa nella forma normale di Chomsky. Lo facciamo per costruzione, scandendo i vari passi.

Passo 1:
Definiamo una nuova variabile iniziale S_0 e aggiungiamo la regola: $S_0 \longrightarrow S$, dove S era la variabile iniziale di G.

Passo 2:
Dobbiamo far sparire tutte le regole espresse nella forma $A \longrightarrow \varepsilon$, dove A è una variabile diversa da quella iniziale. Si fa in due tempi:
1) Eliminiamo tutte le regole nella forma $A \longrightarrow \varepsilon$;
2) Per ogni occorrenza di A nella parte destra della regola, aggiungiamo una nuova regola in cui A non vi compare. In pratica, se abbiamo una regola del tipo $R \longrightarrow uAv$ (dove u e v sono stringhe di variabili e/o terminali), la facciamo diventare $R \longrightarrow uAv/uv$

Passo 3:
Dobbiamo far sparire tutte le regole unitarie espresse nella forma $A \longrightarrow B$. Anche in questo caso si opera in due tempi:
1) Eliminiamo tutte le regole unitarie nella forma $A \longrightarrow B$;
2) Ogni volta che appare la regola $B \longrightarrow u$, aggiungiamo la regola $A \longrightarrow u$

Regola 4:
Concludiamo la costruzione convertendo tutte le altre regole nella forma appropriata. In particolare dovremo individuare ogni regola del tipo:
$A \longrightarrow u_1 u_2 \dots u_k$, con $k \geq 3$ e ogni u_i variabile o terminale
e sostituirla con le regole:
$A \longrightarrow u_1 A_1$
$A_1 \longrightarrow u_2 A_2$
$A_2 \longrightarrow u_3 A_3$
$\dots$
$A_{k-2} \longrightarrow u_{k-1} u_k$, dove le varie A_i sono nuove variabili.

Quando arriveremo ad avere tutte le regole con $k = 2$, sostituiamo il terminale u_i nelle regole precedenti con la nuova variabile U_i, e aggiungiamo la regola: $U_i \longrightarrow u_i$

Esempio di conversione

La grammatica G da convertire è la seguente:

$S \longrightarrow ASA \mid aB$
$A \longrightarrow B \mid S$
$B \longrightarrow b \mid \varepsilon$

Applichiamo il passo 1. Aggiungiamo una nuova variabile iniziale:

$S_0 \longrightarrow S$
$S \longrightarrow ASA \mid aB$
$A \longrightarrow B \mid S$
$B \longrightarrow b \mid \varepsilon$

Applichiamo il passo 2. Eliminiamo la regola $B \longrightarrow \varepsilon$:

$S_0 \longrightarrow S$
$S \longrightarrow ASA \mid aB \mid a$
$A \longrightarrow B \mid S \mid \varepsilon$
$B \longrightarrow b$

Eliminiamo la regola $A \longrightarrow \varepsilon$:

$S_0 \longrightarrow S$
$S \longrightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S$
$A \longrightarrow B \mid S$
$B \longrightarrow b$

Applichiamo il passo 3. Eliminiamo la regola unitaria $S \longrightarrow S$:

$S_0 \longrightarrow S \mid ASA \mid aB \mid a \mid SA \mid AS$
$S \longrightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \longrightarrow B \mid S$
$B \longrightarrow b$

Eliminiamo la regola unitaria $S_0 \longrightarrow S$:

$S_0 \longrightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \longrightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \longrightarrow B \mid S$
$B \longrightarrow b$

Eliminiamo la regola unitaria $A \longrightarrow B$:

$S_0 \longrightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \longrightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \longrightarrow S \mid b$
$B \longrightarrow b$

Eliminiamo la regola unitaria $A \longrightarrow S$:

$S_0 \longrightarrow ASA \mid aB \mid a \mid SA \mid AS$
$S \longrightarrow ASA \mid aB \mid a \mid SA \mid AS$
$A \longrightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS$
$B \longrightarrow b$

Applichiamo il passo 4. Convertiamo le regole rimanenti nella forma appropriata:

$S_0 \longrightarrow AA_1 \mid UB \mid a \mid SA \mid AS$
$S \longrightarrow AA_1 \mid UB \mid a \mid SA \mid AS$
$A \longrightarrow b \mid AA_1 \mid UB \mid a \mid SA \mid AS$
$A_1 \longrightarrow SA$
$U \longrightarrow a$
$B \longrightarrow b$